

Experiment No: 02

Machine Learning

Aim:

Split Data into Train/Validation/Test sets (75-15-15) and save using `train_test_split`.

Objective:

- To learn how to partition a dataset into training, validation, and testing subsets.
- To implement data splitting using `sklearn`'s `train_test_split` function.
- To save the resulting subsets into separate CSV files for future use.

Prerequisites:

- Software: Python 3.x installed with `pandas`, `scikit-learn`, `matplotlib`.
- Hardware: A computer with minimum 4GB RAM.
- Skills: Basic Python programming and familiarity with CSV data handling.

Theory:

In machine learning, datasets must be divided into three distinct parts to ensure proper evaluation and avoid overfitting. The training set (75%) is used to fit the model. The validation set (15%) is used for hyperparameter tuning and model selection. The test set (15%) is used to assess the final model's performance on unseen data. The `train_test_split` function from `sklearn` enables easy and reproducible splitting of datasets. Storing these subsets into separate files ensures reproducibility and easier deployment.

Apparatus/Software Requirements

- Python IDE or Jupyter Notebook.
- Internet connection (for initial library installation).

Procedure:

- ◆ Step 1: Load the dataset (e.g., *Cars_Datasets_2025.csv*) into a `pandas` DataFrame using `pd.read_csv()`. Display the first few rows and shape to ensure it is correctly loaded.
- ◆ Step 2: Separate the input features (X) and the target variable (y). For example, X = "Engines" and "CC/Battery Capacity", while y = "Total Speed".
- ◆ Step 3: Apply `train_test_split` to divide the data into two parts: Training (70%) and Temporary (30%). The training set will be used to build the model.
- ◆ Step 4: Further split the Temporary set equally into Validation (15%) and Test (15%) subsets using another `train_test_split`. The validation set will help in tuning the model, while the test set is reserved for final evaluation.

- ◆ Step 5: Concatenate the features (X) and target (y) back into combined DataFrames for Train, Validation, and Test sets, making them easier to handle.
- ◆ Step 6: Save each of the three subsets (train, val, test) into separate CSV files using `to_csv()` for future use.
- ◆ Step 7: Print and verify the shapes of Train, Validation, and Test sets to confirm the correct 75-15-15 ratio split.

Program Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

df = pd.read_csv("Cars_Datasets_2025.csv", encoding="ISO-8859-1")
print(df.head())
print(df.shape)

X = df[["Engines", "CC/Battery Capacity"]]
y = df["Total Speed"]

# First split: Train (70%) and Temp (30%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30,
                                                    random_state=42)

# Second split: Validation (15%) and Test (15%)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
                                                test_size=0.50, random_state=42)

# Combine features + target for saving
train_set = pd.concat([X_train, y_train], axis=1)
val_set = pd.concat([X_val, y_val], axis=1)
test_set = pd.concat([X_test, y_test], axis=1)

# Save to CSV
train_set.to_csv("train.csv", index=False)
val_set.to_csv("val.csv", index=False)
test_set.to_csv("test.csv", index=False)
```

```
print("Train:", X_train.shape, y_train.shape)
print("Val:", X_val.shape, y_val.shape)
print("Test:", X_test.shape, y_test.shape)
```

Expected Output:

- Display of dataset head and shape.

	Company Names	Cars Names	Engines CC/Battery	Capacity	HorsePower
0	FERRARI	SF90 STRADALE	V8	3990 cc	963 hp
1	ROLLS ROYCE	PHANTOM	V12	6749 cc	563 hp
2	Ford	KA+	1.2L Petrol	1,200 cc	70-85 hp
3	MERCEDES	GT 63 S	V8	3,982 cc	630 hp
4	AUDI	AUDI R8 Gt	V10	5,204 cc	602 hp

- Shapes of train, validation, and test splits printed, e.g.:

Train: (525, 2) (525,)

Val: (112, 2) (112,)

Test: (113, 2) (113,)

- Three CSV files generated:

train.csv

val.csv

test.csv

Observations:

The dataset was successfully split into training, validation, and testing subsets in a 75-15-15 ratio. Each subset was saved into a separate CSV file for later use. This ensures model training, hyperparameter tuning, and evaluation are performed independently, reducing chances of overfitting.

Conclusion:

This experiment demonstrated the importance of splitting datasets into train, validation, and test subsets. Such partitioning allows for fair evaluation of machine learning models and ensures that final performance metrics are unbiased and reliable.

Viva Questions:

1. Why do we split the dataset into training, validation, and test sets?
2. What is the role of `random_state` in `train_test_split`?
3. How can an imbalanced dataset affect the split results?
4. Why is the validation set important for hyperparameter tuning?
5. What might happen if the test set is used during model training?

